DevDoc Studio — SRS status report

Where the application stands against the 52 requirements in the Software Requirements Specification, and what is left before it is deployed. Each requirement built in this round was checked against the code and its tests; the companion spreadsheet *SRS-DevdocStudio-Gap-Analysis.xlsx* carries the same status and notes per row.

52
REQUIREMENTS ASSESSED

52
DONE

0
PARTIALLY DONE

0
NOT STARTED

AREA	PROGRESS	DONE	PARTIAL	NOT STARTED	TOTAL	% DONE
Document editor & UI	11	11	0	0	11	**100%**
Test manager	21	21	0	0	21	**100%**
Release management	14	14	0	0	14	**100%**
Non-functional	6	6	0	0	6	**100%**
All requirements	52	**52**	**0**	**0**	**52**	**100%**

Key points

1. **All 52 requirements are now built (was 38 on 4 Nov).** This round added remote test execution end to end, CI/CD, scheduled runs, trend graphs, Jira / GitHub blockers, connecting existing code, JWT and single sign-on, and HTTPS.
2. **Real test execution works against a defined interface.** Admins register test servers with encrypted credentials; runs are sent there, polled by the server every 5–30 s and pushed live to the page. A real test PC has to implement the two endpoints in docs/test-pc-api.md.
3. **Some integrations are tested against stand-ins only.** Jira, GitHub, the single sign-on provider and the test PC were simulated in tests, and the Jenkins / Bitbucket pipeline files have not run in a real Jenkins or Bitbucket. Each needs one check against the real system.
4. **Before deploying:** set SECRETS_KEY and BRIDGE_TOKEN, install a real TLS certificate, and add a password to the embedded code editor (it still has none, so anyone who can reach the app's address can open it).
5. **Still to check:** the 38 statuses carried over from 4 Nov were not re-checked in this round, and the existing "Import output.xml" button reads Robot results incorrectly (the new CI upload reads them correctly).

Recommended order of work

Every requirement from the 2 Nov plan appears once below; green ones are done. Effort: S = days, M = 1–2 weeks, L = several weeks.

PHASE	FOCUS	REQUIREMENTS (COLOUR = CURRENT STATUS)	WHY IN THIS ORDER
0	**Quick wins & fixes** Effort: S · Done	008 011 007 013 018 015 024 016	Done: admin-only data management with backup and reset, sign-in history, full test case fields, issue IDs on failed results, CSV / XML / JSON import-export, run reports, and runs from picked cases or sections.
1	**Move test management to the backend** Effort: L · Done	012 014 073 031 025	Done: test data is shared by all users; projects and sub-sections, test case history, fast large suites, combined reports, run comparison.
2	**Real test execution engine** Effort: L · Done	019 075 020 021 022 023 072 028 026 027	Done: test servers with encrypted credentials, runs sent over REST, server-side polling (5 / 10 / 30 s) with progress, live updates, trend graphs, scheduled runs, CI tokens with Jenkins and Bitbucket pipeline files.
3	**Release management** Effort: L · Done	037 038 039 040 035 034 042 043 044 045 041 033 046 036	Done: releases and milestones, RC builds with image details, linked test runs, readiness checks, sign-offs, promotion to Final, release notes, audit log, and blockers read from Jira / GitHub (044).
4	**Security & platform hardening** Effort: M · Done	074 076 071 030	Done: HTTPS with TLS 1.3 only, JWT access tokens and OpenID Connect single sign-on, fast navigation, connecting existing Git repositories.

Chip colours: done partial not started . All five phases are built; what remains is checking the integrations against real systems and deploying.

Document Editor & UI

DDS-SRS-001 – 011 · 11 done · 0 partial · 0 not started

#	REQ ID	REQUIREMENT	STATUS	WHAT EXISTS TODAY / WHAT IS MISSING
1	DDS-SRS-001	**List in-progress documents; open in editor**	Done	Users see documents in Draft or In Review and can open them in the editor.
2	DDS-SRS-002	**Users edit only their own documents**	Done	Users can edit and delete only documents they wrote.
3	DDS-SRS-003	**Admin views all documents**	Done	Admins see all documents, in any status, and can open them in the editor.
4	DDS-SRS-004	**Admin edits all documents**	Done	Admins can edit and delete all documents.
5	DDS-SRS-005	**Dashboard landing page with document stats**	Done	After login the dashboard shows total, approved, in-review and draft document counts and the latest Google Drive upload.
6	DDS-SRS-006	**Sidebar entry for the document editor**	Done	The sidebar has a Document Editor entry.
7	DDS-SRS-007	**Sidebar: doc editor, data management, code editor**	Done	The sidebar has Document Editor, Coder (code editor) and, for admins, Data Management.
8	DDS-SRS-008	**Data management restricted to admins**	Done	Only admins can open Data Management. It lets them download a backup of all data and reset all data after typing a confirmation.
9	DDS-SRS-009	**Save document to the VS Code project folder**	Done	A document's .md file is saved into the code editor's project folder; the .docx and .md are also downloaded. If the editor is unreachable, they are only downloaded.
10	DDS-SRS-010	**Download documents**	Done	Documents download as .docx, .md or JSON, and can be exported to Google Docs.
11	DDS-SRS-011	**Admin audit log of user sessions**	Done	Admins see each user's sign-in and sign-out times, role and session length, for all users.

Test Manager

DDS-SRS-012 – 032 · 21 done · 0 partial · 0 not started

#	REQ ID	REQUIREMENT	STATUS	WHAT EXISTS TODAY / WHAT IS MISSING
12	DDS-SRS-012	**Hierarchy: Projects > Suites > Sections > Cases**	Done	Projects contain suites, suites contain sections and sub-sections, sections contain test cases.
13	DDS-SRS-013	**Test case fields (ID, steps, post-conditions, attachments)**	Done	Each case has an auto-generated ID (TC-001, TC-002, ...), title, pre-conditions, type, priority (Low / Medium / High / Critical), steps with expected results, post-conditions and attachments (screenshots, log files).
14	DDS-SRS-014	**Revision history for test cases**	Done	Every change to a test case is kept as a revision (who, when, what changed); earlier revisions can be viewed and restored.
15	DDS-SRS-015	**Import/export test cases as CSV, XML, JSON**	Done	Test cases can be imported and exported as CSV, XML and JSON.
16	DDS-SRS-016	**Create test runs from cases or whole suites**	Done	Runs can be created from a whole suite, from picked test cases or whole sections and sub-sections, or from a filter (section, priority, type). Cases in the bin are left out.
17	DDS-SRS-017	**Manual execution statuses**	Done	Testers set each case in a run to Passed, Failed, Blocked, Retest, Untested or Skipped.
18	DDS-SRS-018	**Attach Jira/GitHub issue IDs to failed results**	Done	Jira or GitHub issue IDs can be attached to each failed result in a run.
19	DDS-SRS-019	**Register remote test servers (URL, auth, payload)**	Done	Admins register remote test servers under Data Management: endpoint URL, authentication (none, API key, Basic or OAuth2 client credentials) and payload parameters. A Test button checks the connection.
20	DDS-SRS-020	**Trigger remote runs from the UI via REST**	Done	'Run Test' asks which registered test server to use; the server then starts the run there over its REST API (docs/test-pc-api.md). A simulated test PC remains for demos. Not yet run against a real test PC.
21	DDS-SRS-021	**Backend polling service (interval, states, metrics)**	Done	A background job in the API polls each active remote run every 5, 10 or 30 seconds (set per server) and records Queued, Running, Completed or Failed, with total, completed, passed and failed counts and completion %.
22	DDS-SRS-022	**Real-time UI updates without refresh**	Done	The server pushes each change in a remote run to the page as it happens (server-sent events). If the stream drops, the page checks every 5 seconds instead.

Test Manager (continued)

DDS-SRS-012 – 032 · 21 done · 0 partial · 0 not started

#	REQ ID	REQUIREMENT	STATUS	WHAT EXISTS TODAY / WHAT IS MISSING
23	DDS-SRS-023	**Metrics dashboard (completion %, donut, velocity)**	Done	Each run shows a pass/fail donut and pass %. Reports has an execution-velocity chart (test cases executed per day) and a pass-rate trend across finished runs, filtered by range and suite, with a table view.
24	DDS-SRS-024	**Downloadable summary reports (PDF, HTML, CSV)**	Done	Each run's summary report downloads as HTML or CSV, or opens ready to print for saving as PDF.
25	DDS-SRS-025	**Compare runs to detect regressions**	Done	A run can be compared with any earlier run of the same suite, listing regressions, fixed, still-failing, new and removed cases.
26	DDS-SRS-026	**Run tests locally and in CI/CD**	Done	Tests run locally in the code editor, or from a CI/CD pipeline: the pipeline runs a suite on a test server, or runs Robot itself and uploads output.xml (ci/devdoc-ci.sh).
27	DDS-SRS-027	**Jenkins and Bitbucket integration**	Done	Jenkins and Bitbucket Pipelines call DevDoc Studio with a CI token; a ready-made Jenkinsfile and bitbucket-pipelines.yml are in docs/ci. Each run records its build, branch and commit. Not yet run in a real Jenkins or Bitbucket.
28	DDS-SRS-028	**Scheduled runs**	Done	Runs can be scheduled once, daily or weekly on a registered test server. The server starts them by itself and fills in the results; no browser needs to be open.
29	DDS-SRS-029	**Integrated code editor with AI generation**	Done	The Coder screen embeds a code editor with an AI agent; test cases and Robot scripts can be generated with AI from a technical analysis document.
30	DDS-SRS-030	**Connect existing code**	Done	The Coder screen's Repositories panel clones an existing Git repository into the code editor's workspace, and pulls or removes it. Private repositories use an access token, stored encrypted.
31	DDS-SRS-031	**Consolidated manual + automated report**	Done	A run's report combines manual and automated results, with totals for each.
32	DDS-SRS-032	**Share docs/test cases with the AI code editor**	Done	Documents and test suites can be sent to the AI code editor's project folder.

Release Management

DDS-SRS-033 – 046 · 14 done · 0 partial · 0 not started

#	REQ ID	REQUIREMENT	STATUS	WHAT EXISTS TODAY / WHAT IS MISSING
33	DDS-SRS-033	**Create releases, approve RC>Final, override gates, sign off**	Done	Release managers and admins create releases, override failing readiness checks with a reason and promote an RC to Final; each assigned user signs off their role.
34	DDS-SRS-034	**Link releases to test runs; log blocking defects**	Done	Test runs are linked to each RC build and blocking issues are logged against the release; each build shows Pending, Verified or Failed from its linked runs.
35	DDS-SRS-035	**Register build artifacts (checksums, URLs)**	Done	Each RC build records its artifact URL, repository and SHA-256 checksum.
36	DDS-SRS-036	**View release status, metrics and release notes**	Done	Each release shows its status, milestones done, builds, latest pass rate, open blockers and Final build, and has release notes that can be drafted from the release.
37	DDS-SRS-037	**Create a release with version, type, date, status**	Done	Releases are created with version, type (Major / Minor / Patch / Hotfix), target date, customer segment and status (Planned, In Progress, RC Frozen, Approved, Deployed, Cancelled).
38	DDS-SRS-038	**Release phase milestones (freeze ... GA)**	Done	Each release has Code Freeze, RC Build Generation, QA Regression Window, Final Sign-off and GA milestones, with planned and actual dates.
39	DDS-SRS-039	**Multiple RC builds per release**	Done	RC builds are numbered in sequence per release (e.g. v2.4.0-RC100, v2.4.0-RC101).
40	DDS-SRS-040	**Image details per build (tag, SHA, URL, checksum)**	Done	Each build records image name / tag, commit SHA, branch / tag, artifact URL, repository, SHA-256, build time and CI/CD build ID.
41	DDS-SRS-041	**Promote an RC image to Final**	Done	An RC can be promoted to Final only when its readiness checks pass (or were overridden) and every required role approved; one Final build per release.
42	DDS-SRS-042	**Link test runs to RC images**	Done	One or more test runs can be linked to each RC build; the build shows what share of the linked runs' results were executed.
43	DDS-SRS-043	**Automatic release-readiness gates**	Done	Readiness is worked out per build from its linked runs: minimum pass rate (default 98%, editable), no open Critical or Blocker issues, and chosen mandatory suites fully executed.
44	DDS-SRS-044	**Link Jira/GitHub release blockers**	Done	Each release can name its blockers as a Jira JQL query or a GitHub repository and label; matching issues are read from the tracker every 5 minutes with title, link, severity and status. Issues can also be linked by key. Tested against simulated Jira and GitHub only.
45	DDS-SRS-045	**Multi-role electronic sign-offs**	Done	Each release lists the roles that must sign off (default QA Lead, Dev Lead, Product Owner) and who holds each; only that person can approve or reject, and all must approve before promotion.
46	DDS-SRS-046	**Immutable audit log for release actions**	Done	Every release action (status changes, builds, linked runs, overrides, sign-offs, promotions, blockers) is logged with who and when. Entries cannot be edited or deleted in the app, and a change made directly in storage is flagged.

Non-functional requirements

DDS-SRS-071 – 076 · 6 done · 0 partial · 0 not started

#	REQ ID	REQUIREMENT	STATUS	WHAT EXISTS TODAY / WHAT IS MISSING
71	DDS-SRS-071	**UI navigation under 1.5 s**	Done	Every screen opened in under 0.4 seconds, measured on a production build on the development machine.
72	DDS-SRS-072	**Polling supports 100 concurrent runs**	Done	100 active runs were all polled within one 1-second cycle (about 0.2 seconds), with status reads of about 1 ms, measured on the development machine against a simulated test server.
73	DDS-SRS-073	**5,000+ case suites load in under 500 ms**	Done	A suite with 5,000 test cases opened in about 0.3 seconds, measured on the development machine; cases are looked up by suite, not searched one by one.
74	DDS-SRS-074	**HTTPS / TLS 1.3 in transit**	Done	The Docker setup serves the app, API and code editor over HTTPS with TLS 1.3 only (TLS 1.2 is refused) and redirects plain HTTP; database and service ports are reachable only from the machine itself. A deployment needs a real certificate.
75	DDS-SRS-075	**Server API keys encrypted at rest (AES-256)**	Done	Test server, Jira / GitHub and repository credentials are encrypted with AES-256-GCM under a server key (SECRETS_KEY) and never returned by the API.
76	DDS-SRS-076	**JWT and OAuth2/SSO authentication**	Done	Users can sign in with email and password or single sign-on through any OpenID Connect provider (Entra ID, Okta, Google, Keycloak), for existing accounts only. JWT access tokens work on every API route and are revoked on sign-out. Tested against a simulated provider only.

Decisions

#	DECISION	WHERE IT STANDS
1	**Background job queue**	Decided: a background job inside the API (no separate queue), with Redis locks so several API instances never repeat work. Polls runs, fires schedules, syncs blockers.
2	**Test PC / hardware API**	Defined: docs/test-pc-api.md (start a run, report a run). The real test PC needs to implement it.
3	**Jira, Jenkins, Bitbucket access**	Built against their public APIs; still needed: an account on each to try it against the real systems.
4	**SSO provider**	Built for any OpenID Connect provider (Entra ID, Okta, Google, Keycloak); still to choose which one, and register the app with it.
5	**JWT vs. sessions**	Decided: both. The browser keeps the session cookie; scripts and tools use JWTs, which are revoked when their session signs out.
6	**Priority scale**	Decided: Low / Medium / High / Critical (013).
7	**Deployment target**	Still needed, for a real TLS certificate and the server's secrets. HTTPS itself is built.